

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import GradientBoostingRegressor

from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

import joblib
```

```
from google.colab import files
```

```
uploaded = files.upload()
```

data.csv

data.csv(text/csv) - 526795 bytes, last modified: 5/8/2026 - 100% done
Saving data.csv to data.csv

```
df = pd.read_csv("data.csv")
```

```
df.head()
```

	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront
0	2014-05-02 00:00:00	313000.0	3.0	1.50	1340	7912	1.5	
1	2014-05-02 00:00:00	2384000.0	5.0	2.50	3650	9050	2.0	
2	2014-05-02 00:00:00	342000.0	3.0	2.00	1930	11947	1.0	
3	2014-05-02 00:00:00	420000.0	3.0	2.25	2000	8030	1.0	
4	2014-05-02 00:00:00	550000.0	4.0	2.50	1940	10500	1.0	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4600 entries, 0 to 4599
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   date                  4600 non-null  object
1   price                 4600 non-null  float64
2   bedrooms              4600 non-null  float64
3   bathrooms             4600 non-null  float64
4   sqft_living           4600 non-null  int64
5   sqft_lot              4600 non-null  int64
6   floors                4600 non-null  float64
7   waterfront            4600 non-null  int64
8   view                  4600 non-null  int64
9   condition             4600 non-null  int64
10  sqft_above            4600 non-null  int64
11  sqft_basement         4600 non-null  int64
12  yr_built              4600 non-null  int64
13  yr_renovated          4600 non-null  int64
14  street                4600 non-null  object
15  city                  4600 non-null  object
16  statezip              4600 non-null  object
17  country               4600 non-null  object
dtypes: float64(4), int64(9), object(5)
memory usage: 647.0+ KB
```

df.describe()

	price	bedrooms	bathrooms	sqft_living	sqft_lot	
count	4.600000e+03	4600.000000	4600.000000	4600.000000	4.600000e+03	4600
mean	5.519630e+05	3.400870	2.160815	2139.346957	1.485252e+04	1
std	5.638347e+05	0.908848	0.783781	963.206916	3.588444e+04	0
min	0.000000e+00	0.000000	0.000000	370.000000	6.380000e+02	1
25%	3.228750e+05	3.000000	1.750000	1460.000000	5.000750e+03	1
50%	4.609435e+05	3.000000	2.250000	1980.000000	7.683000e+03	1
75%	6.549625e+05	4.000000	2.500000	2620.000000	1.100125e+04	2
max	2.659000e+07	9.000000	8.000000	13540.000000	1.074218e+06	3

```
df.isnull().sum()
```

	0
date	0
price	0
bedrooms	0
bathrooms	0
sqft_living	0
sqft_lot	0
floors	0
waterfront	0
view	0
condition	0
sqft_above	0
sqft_basement	0
yr_built	0
yr_renovated	0
street	0
city	0
statezip	0
country	0

dtype: int64

```
le = LabelEncoder()

for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])

df.head()
```

	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterf
0	0	313000.0	3.0	1.50	1340	7912	1.5	
1	0	2384000.0	5.0	2.50	3650	9050	2.0	
2	0	342000.0	3.0	2.00	1930	11947	1.0	
3	0	420000.0	3.0	2.25	2000	8030	1.0	
4	0	550000.0	4.0	2.50	1940	10500	1.0	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
plt.figure(figsize=(12,8))

sns.heatmap(df.corr(), annot=True)

plt.show()
```

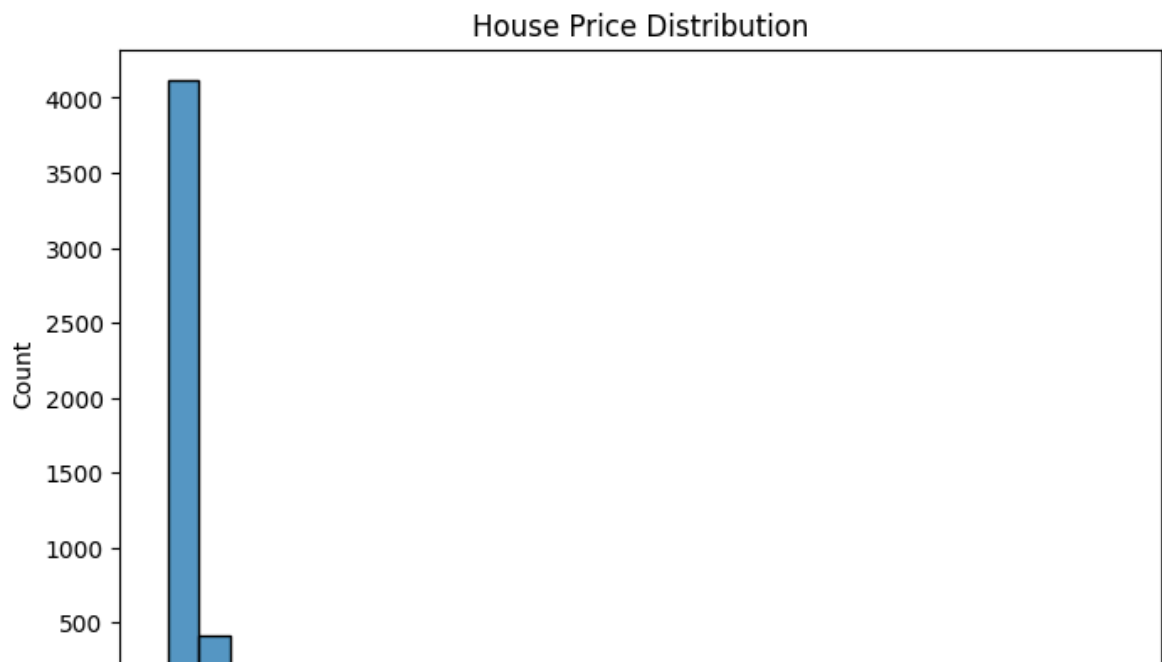


```
plt.figure(figsize=(8,5))

sns.histplot(df['price'], bins=30)

plt.title("House Price Distribution")
```

```
plt.show()
```



```
X = df.drop('price', axis=1)
```

```
y = df['price']
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled,  
    y,  
    test_size=0.2,  
    random_state=42  
)
```

```
lr_model = LinearRegression()
```

```
lr_model.fit(X_train, y_train)
```

```
lr_pred = lr_model.predict(X_test)
```

```
rf_model = RandomForestRegressor()
```

```
rf_model.fit(X_train, y_train)
```

```
rf_pred = rf_model.predict(X_test)
```

```
gb_model = GradientBoostingRegressor()
```

```
gb_model.fit(X_train, y_train)
```

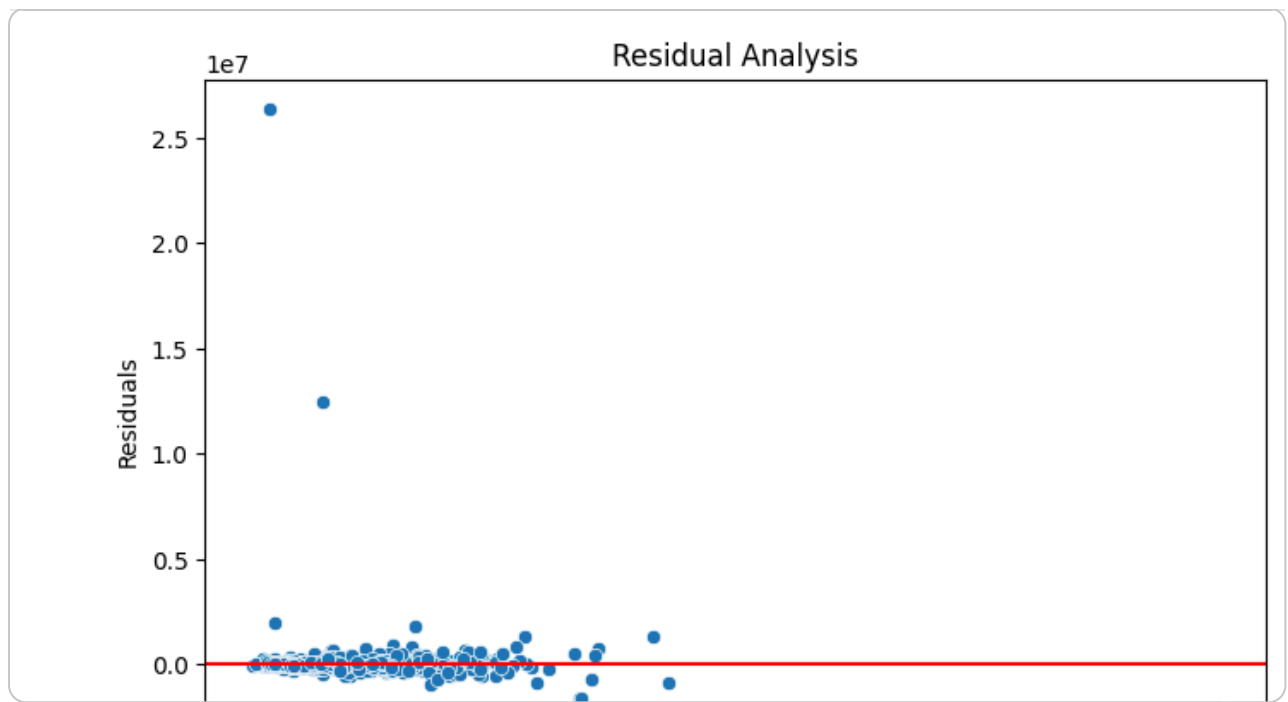
```
gb_pred = gb_model.predict(X_test)
```

```
def evaluate_model(name, y_test, pred):  
  
    mae = mean_absolute_error(y_test, pred)  
  
    rmse = np.sqrt(mean_squared_error(y_test, pred))  
  
    r2 = r2_score(y_test, pred)  
  
    print(name)  
    print("MAE:", mae)  
    print("RMSE:", rmse)  
    print("R2 Score:", r2)  
    print("-----")
```

```
evaluate_model("Linear Regression", y_test, lr_pred)  
  
evaluate_model("Random Forest", y_test, rf_pred)  
  
evaluate_model("Gradient Boosting", y_test, gb_pred)
```

```
Linear Regression  
MAE: 207816.82941665527  
RMSE: 993048.575350367  
R2 Score: 0.03304504390209173  
-----  
Random Forest  
MAE: 166618.06276434692  
RMSE: 988094.69339019  
R2 Score: 0.042668405298577405  
-----  
Gradient Boosting  
MAE: 170696.95694904664  
RMSE: 984935.2607158577  
R2 Score: 0.04878075287429906  
-----
```

```
residuals = y_test - rf_pred  
  
plt.figure(figsize=(8,5))  
  
sns.scatterplot(x=rf_pred, y=residuals)  
  
plt.axhline(y=0, color='r')  
  
plt.xlabel("Predicted Prices")  
  
plt.ylabel("Residuals")  
  
plt.title("Residual Analysis")  
  
plt.show()
```



```
joblib.dump(rf_model, "house_price_model.pkl")
```

```
print("Model Saved Successfully")
```

Model Saved Successfully

```
sample = X_test[0].reshape(1, -1)
```

```
prediction = rf_model.predict(sample)
```

```
print("Predicted House Price:", prediction[0])
```

Predicted House Price: 440346.01

```
files.download("house_price_model.pkl")
```